

**Computer Science and Engineering Department, SVNIT, Surat**  
**M.Tech. I – Semester I**  
**Design and Analysis of Algorithm (CSDS103)**  
**Lab Assignment 4**  
**Divide and Conquer**

**General Instructions**

1. Develop the algorithm/pseudocode before implementing the program.
2. Analyse the time and auxiliary space complexity of the proposed approach.
3. Implement the required algorithms and compare their performance for different input sizes.
4. Record execution times and use suitable graphs to compare experimental and theoretical behaviour.

**Problem 1 – Sorting and Counting Inversions**

Given an array of  $n$  integers, an inversion is a pair  $(i, j)$  such that

$$i < j \quad \text{and} \quad A[i] > A[j].$$

For example, the array

$$[8, 4, 2, 1]$$

contains six inversions.

Develop a solution based on **Divide and Conquer** that sorts the array and counts the total number of inversions. The sorting and inversion-counting should be performed together rather than first sorting the array and then searching for inversions.

Also implement a straightforward approach that examines pairs of elements to count inversions.

1. Analyse the time and auxiliary space complexity of both approaches.
2. Generate arrays of different sizes and measure the execution time of both approaches.
3. Compare the experimental results with the theoretical complexities.
4. Compare the performance of the Divide and Conquer sorting approach with at least two other sorting algorithms studied in the previous laboratory.
5. Generate suitable performance graphs showing the relationship between input size and execution time.

**Problem 2 – Fast Exponentiation**

Given two integers  $a$  and  $n$ , compute

$$a^n$$

for a large value of  $n$ .

Implement and compare the following approaches:

1. Repeated multiplication.
2. Recursive exponentiation based on the relation

$$a^n = \begin{cases} 1, & n = 0, \\ a^{n/2} \times a^{n/2}, & n \text{ is even,} \\ a \times a^{n-1}, & n \text{ is odd.} \end{cases}$$

3. A Divide and Conquer approach that computes  $a^{n/2}$  only once and reuses the result.

Measure the execution time for different values of  $n$  and compare the growth of the three approaches.

1. Derive the recurrence relation for the Divide and Conquer approach.
2. Determine the time and auxiliary space complexity of each approach.
3. Generate a performance graph of **Input Size vs Execution Time**.
4. Compare the experimental results with the theoretical complexities.
5. Explain the effect of repeatedly computing the same subproblem in the recursive approach.

### Analysis:

Consider very large values of  $n$ . Explain why reducing the number of multiplications can significantly affect the execution time. Discuss whether the recursive Divide and Conquer solution can be implemented iteratively and what effect this would have on auxiliary space.

Compare the two problems from the perspective of the Divide and Conquer paradigm. Explain how a large problem is divided into smaller subproblems, how the solutions are combined, and how this affects the overall time complexity. Discuss the difference between theoretical complexity and experimentally observed execution time.